Qubifide: A BB84-Based Framework for Secure Communication and Eavesdropping Analysis

PROJECT REPORT

Submitted in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

Department of Computer Science and Engineering

Submitted to

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY BHOPAL (M.P.)

Submitted by

Jaswant Swaram (24U020113 Vth)

Akshat Kumar (24U020138 Vth)

Under the supervision of

Dr. Supriyo Mandal

Assistant Professor

(Computer Science & Engineering)

November 2026

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY BHOPAL (M.P.)



CERTIFICATE

This is to certify that the work embodied in this report entitled **"Qubifide: A BB84-Based Framework for Secure Communication and Eavesdropping Analysis"** has been satisfactorily completed by **STUDENT NAME (SCHOLAR NO.)** It is a bonafide piece of work, carried out under our guidance in the **Department of Computer Science and Engineering**, **Indian Institute of Information Technology, Bhopal** for the partial fulfillment of the Bachelor of Engineering during the academic year 202_-__.

Date:

Dr. Supriyo Mandal
Assisstant Professor
Department of CSE,
IIIT Bhopal (M.P.)

Dr. Rahul Chaurasia
Department of CSE,
IIIT Bhopal (M.P.)

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY BHOPAL (M.P.)

DECLARATION

We hereby declare that the following major project synopsis entitled "**Qubifide: A BB84-Based Framework for Secure Communication and Eavesdropping Analysis**" presented in the is the partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in** Computer Science and Engineering. It is an authentic documentation of our original work carried out under the able guidance of **Dr. Supriyo Mandal**. The work has been carried out entirely at the Indian Institute of Information Technology, Bhopal. The project work presented has not been submitted in part or whole to award of any degree or professional diploma in any other institute or organization.

We, with this, declare that the facts mentioned above are true to the best of our knowledge. In case of any unlikely discrepancy that may occur, we will be the ones to take responsibility.

Jaswant Swaram (24U020113)
Akshat Kumar (24U020138)

ACKNOWLEDGEMENT

We, Jaswant Swaram and Akshat Kumar, would like to express our sincere gratitude to everyone who supported and guided us throughout the successful completion of our minor project titled "Qubifide: A BB84-Based Framework for Secure Communication and Eavesdropping Analysis ".

First and foremost, we would like to extend our heartfelt thanks to our esteemed **Director**, Dr. Ashutosh Kumar Singh, for providing us with the necessary resources and facilities for conducting our project. His encouragement and vision played a key role in helping us accomplish our goals.

We are deeply grateful to our **Project Guide**, **Dr. Supriyo Mandal**, for his/her constant guidance, invaluable suggestions, and expert advice throughout the project. His/her knowledge and insights helped us enhance the quality of our work, and his/her support kept us motivated during difficult times.

Our sincere thanks to the **Department Coordinator**, **Dr. Rahul Chaurasia,** for the valuable support and encouragement provided during completion of the project.

We would also like to express our gratitude to all the **faculties** of the **Computer Science & Engineering** department for their valuable guidance and support in clarifying doubts and providing us with the right direction throughout the smooth execution of our project.

A special thanks to our **friends** for their continuous support, valuable suggestions, and helping us during various stages of the project. Their insights and encouragement motivated us to keep working on the project with enthusiasm.

Last but certainly not the least, we would like to express our deepest gratitude to our **parents** and family members for their love, patience, and continuous moral support throughout the course of this project. Their understanding and encouragement were the driving force behind our efforts.

We are truly thankful to everyone who directly or indirectly contributed to the success of this project. Without their assistance and support, this project would not have been possible.

Jaswant Swaram
Akshat Kumar
24U020113
24U020138

AREA OF WORK

This project looks into the areas of quantum computing, mathematics, cryptography and software development. The aim is to show how this simulated quantum key distribution can be used alongside traditional cryptography techniques to produce secure communications.

This project looks into the area of quantum cryptography. The BB84 Quantum Key Distribution protocol has been implemented using Python, Qiskit and Qiskit Aer. This includes simulating the preparation of quantum states, measurement of quantum states, basis matching and sifting of keys. The probability theory, linear algebra and Quantum Bit Error Rate concepts have been used to analyse the effect of a potential eavesdropper on the quantum channel.

This project also looks into the area of cryptography. The HKDF and AES-GCM standards have been used to generate the key encrypting key and set-up an authenticated encryption with associated data. This uses cryptographic primitives to ensure secure key exchange.

This project also looks into the area of software development. The project uses the React and TypeScript programming languages for the client-side. The project uses Python and FastAPI for the server-side. This project has produced a web application which allows 2 parties to securely communicate using a quantum key exchange. The application allows for session management, key exchange analysis and encrypted message exchange.

TABLE OF CONTENT

S.no	Title	Page No.
	Certificate	
	Declaration	
	Abstract	
1	Introduction	1
2	Literature review or Survey	2
3	Methodology & Work Description	3
4	Proposed algorithm	4
5	Proposed flowchart/ DFD/ Block Diagram	5
6	Tools & Technology Used	6
7	Implementation & Coding	7
8	Result Analysis	8
9	Conclusion & Future Scope	9
10	References	10

LIST OF FIGURES

Fig	Description	Page no.
1	System architecture	11
2	Block diagram of system	18
3	Workflow of system	19
4	Data flow diagram	19
5	Completed Secure Key Exchange	22
6	Completed Message Transfer	22
7	Eve interception	23

LIST OF TABLES

Table No	Description	Page no.
1	Comparative Summary of Existing Research	7
2	QBER-Based Eavesdropping Analysis and Exchange Verification	25
3	Key Exchange Verification and Acceptance	25
4	Secure Message Encryption and Authentication Testing	26

ABSTRACT

Secure communication systems require strong protocols to securely distribute keys, detect eavesdropping and encrypt messages. This project, entitled **"QubiFide: A BB84-Based Framework for Secure Communication and Eavesdropping Analysis"** demonstrates how quantum key distribution can be used alongside traditional cryptography in a software based prototype.The framework utilises the BB84 protocol by implementing it in Qiskit and Qiskit Aer. This allows secure key exchange through quantum state preparation, measurement and basis alignment. The Quantum Bit Error Rate (QBER) is calculated to determine the error rate during transmission. An optional intercept resend attack can be added to show the effects of eavesdropping. Once the keys have been verified, HKDF is used to derive the keys. AES-GCM is used to allow authenticated encryption. The application is built using React and TypeScript for the frontend, with Python and FastAPI for the backend.QubiFide allows the user to gain an understanding of quantum key distribution, eavesdropping analysis and secure message exchange. It is a prototype for a simulation and should not be assumed to be secure in real situations. No actual quantum hardware is used.

Keywords: Quantum Key Distribution (QKD), BB84 Protocol, Quantum Cryptography, Quantum Bit Error Rate (QBER), Eavesdropping Detection, Intercept-Resend Attack, HKDF, AES-GCM, Qiskit, Secure Communication.

GitHub source link: https://github.com/akshatshaw85/QubiFide

1. INTRODUCTION

To ensure secure communication of confidential information such as scientific research, technical documentation, research data and reports, encryption methods have been employed. Traditional encryption methods utilise encryption algorithms and secret keys. However, a challenge remains with creating and managing secure keys. Quantum Key Distribution (QKD) offers an alternative method of securely generating keys utilising quantum mechanical processes. The BB84 protocol introduced by Bennett and Brassard (1984) utilises quantum states being prepared and measured in different bases[1]. This allows key material to be created between the parties involved. Certain types of eavesdropping can cause noticeable disruptions. BB84 utilises mathematics such as probability, linear algebra, quantum states and measurement. Software can be used to simulate these processes allowing the operations to be studied.

1.2 Need For Study

BB84 requires quantum states to be prepared and measured, followed by basis reconciliation and key sifting. Studying these processes can help to understand how quantum measurement relates to secure communication. This project includes a simulated BB84 protocol which can optionally include an intercept resend attack. This compares the key exchange under normal conditions and when being intercepted, to see how eavesdropping could affect the QBER [5], [12]. As QKD isn't directly encrypting messages in an application, HKDF has been included to derive keys for the application. AES-GCM has been included to allow encryption of messages.

1.3 Overview Of QubiFide

QubiFide (A BB84-Based Framework for Secure Communication and Eavesdropping Analysis) is a software prototype that simulates QKD, eavesdropping and secure communication. The system prepares and measures quantum states using Python, Qiskit and Qiskit Aer [6], [20]. After basis reconciliation and key sifting, a sample of the remaining bits will be compared to determine the QBER. The system can optionally include the intercept resend attack. The application uses React and TypeScript for the frontend and Python and FastAPI for the backend. Once the key exchange has been accepted using the simulated key, the system will use HKDF and AES-GCM to derive keys [7], [8]. This provides an interactive platform to understand how quantum simulation can relate to traditional cryptography.

1.4 Aim

The main aim of this project is to demonstrate how BB84 quantum key distribution, QBER disruption analysis and authenticated encryption can be combined into a web application [1], [4], [8]. The project includes quantum state preparation and measurement, basis reconciliation, key sifting, an optional intercept resend attack, key derivation at the application level and secure message exchange. All quantum processes will be simulated using software.
The prototype has been created for educational and experimental purposes only. It will not provide any formal security assurances that would be provided by a full QKD system. More

work would need to be done on secure key exchange, privacy amplification, key management and independent security assessment before the system could be deployed [4], [17], [18].

1.5 Report Structure

This report will include a Literature Review, Problem Statement & Objectives, Proposed Methodology, Algorithms, System Diagrams, Tools, Technologies and Implementation. The Discussion and Results section will include the discussion and analysis of the experimental results. The final chapter will provide a summary of the project and potential areas for future development.

2. LITERATURE REVIEW

Quantum cryptography was developed to help secure key distribution using principles from quantum mechanics[1], [9]. In 1984 Bennett and Brassard developed the BB84 protocol which showed how a shared secret key could be distributed using quantum states in different bases[1].. The main principle behind this protocol is that measuring a quantum state [1], [3], [4] in the wrong basis will cause disturbance to the quantum state. Research has been performed to understand the theory behind QKD and the requirements to implement QKD [4], [18].. Shor and Preskill provided an important proof for the security of BB84 as they related it to entanglement purification and quantum error correcting codes[3]. Scarani et al provided an overview of both the theory and practical implementations of QKD[4]. This shows the importance of having authenticated classical communications, correct key handling and taking into consideration any imperfections[4], [16], [17]. This shows that there is a solid theory behind the BB84 protocol. However it also shows that to create a secure communication system additional classical processing and cryptography is required[4], [17]. QubiFide takes advantage of these principles by building a software simulation of the BB84 protocol [1], [4].

2.1 Mathematical Background of BB84

Two bases are used to represent classical bits using quantum states. The computational basis uses the states $|0\rangle$ and $|1\rangle$ and the diagonal basis uses the superposition states $|+\rangle$ and $|-\rangle$. These are defined as:

$$|+\rangle = \frac{|0\rangle+|1\rangle}{\sqrt{2}} \tag{2.1.1}$$

$$|-\rangle = \frac{|0\rangle-|1\rangle}{\sqrt{2}} \tag{2.1.2}$$

The quantum states have equal probability amplitudes however they have different relative phase factors. We can use the Hadamard gate to transform between the computational and diagonal bases:

$$H = \frac{1}{\sqrt{2}}\begin{bmatrix}1 & 1\\ 1 & -1\end{bmatrix} \tag{2.1.3}$$

So,

$$H|0\rangle = |+\rangle \tag{2.1.4}$$

And

$$H|1\rangle = |-\rangle \tag{2.1.5}$$

This provides a way to represent the preparation and measurements using quantum circuits. The probability of getting a measurement outcome is given using the Born rule:

$$P(m \mid \psi) = |\langle m \mid \psi \rangle|^2 \quad (2.1.6)$$

Where $|\psi\rangle$ is the quantum state and $|m\rangle$ is the measurement outcome. The probability depends on the overlap between the measurement outcome and quantum state. This is why measuring in an incorrect basis can cause uncertainty [1], [9]...During BB84, the sender and receiver randomly decide which basis to prepare and measure. Once the quantum states are sent they compare which bases were used. They will keep the positions where the bases match. This is known as basis reconciliation and key sifting. If the sender and receiver randomly choose between two bases then the expected number of sifted positions out of N qubits sent will be:

$$E[M] = \frac{N}{2} \quad (2.1.7)$$

Where M is the number of positions where the bases match. This is an expected value as the bases are randomly chosen. [1], [3] This shows that there is a mathematical background to BB84. This includes probability theory, linear algebra, quantum states and measurements. This can be used to simulate the preparation of quantum states, measurement outcomes and key sifting.

2.2 Key Processing and Security

Shor and Preskill have shown that BB84 can be analysed through its relationship with entanglement purification and quantum error correcting codes[3]. This provides a theoretical background; however it does not mean that any software implementation using similar circuits to BB84 will follow the assumptions. Once the key has been sifted the parties will have correlated bit strings however the bit strings may have errors. These errors can be corrected using error correction. Privacy amplification can also be used to reduce the amount of information an adversary would know about the key. The sifted key should not be assumed to be a secure cryptographic key [4], [17]..

Authentication is important. The classical communications used during basis reconciliation and protocol should be authenticated to ensure that no one can impersonate a party or alter the protocol [4], [17]. Real QKD systems should take into account any imperfections in the channel. [4], [16]. This is relevant to our application as QubiFide only shows the main stages of a simulated BB84 protocol. The acceptance criteria is a decision made at an implementation level. Error correction, privacy amplification, authenticated protocol negotiation and thorough security analysis are not performed.

2.3 Quantum Bit Error Rate

The Quantum Bit Error Rate (QBER) determines how many of the matching bits in the sampled sifted key do not match. If \(e\) errors are found out of \(k\) compared bits then you can estimate the QBER to be:

$$\hat{Q} = \frac{e}{k} \times 100\% \quad (2.3.1)$$

The higher the QBER the more disagreement there is in the sampled outcomes. This can be due to eavesdropping, noise in the channel or imperfections in the implementation. This makes it useful to know the QBER when analysing the security. However it is not an independent indicator of why the error occurred [4], [5]. An important type of eavesdropping is the intercept resend attack. This involves an adversary (Eve) intercepting the quantum state being sent. The quantum state is measured in a randomly chosen basis. Eve then prepares a new quantum state depending on her measurement outcome. If Eve measures in the incorrect basis this can cause disturbance to the quantum state meaning there will be an error in the measurement done by the receiver. Assuming the bases are chosen randomly and equally and all qubits are intercepted then the expected QBER of the sifted bits will be:

$$Q_{\mathrm{IR}} = \frac{1}{2} \times \frac{1}{2} = \frac{1}{4} = 25\% \tag{2.3.2}$$

The first term is the probability Eve will choose an incorrect basis and the second term is the probability of having an error in the bit [1], [4], [12]. This gives us an expected error rate. Lee, Sohn, and Leo looked at statistically detecting eavesdropping in BB84 using QBER thresholds [5]. They looked at the balance between accuracy and quantum resources. This shows that the decision to detect eavesdropping depends on the sample size. If a small sample is taken this may not give an accurate representation of the error rate. However if a larger sample is taken generally more information can be gained. QubiFide can be used to compare the usual exchange to one with the intercept resend attack turned on. This allows the QBER to be estimated and shows how the simulation behaves. However the 25% expected value will not always be found in all finite simulations. A low QBER doesn't guarantee that there is no eavesdropper [4], [5].

2.4 Quantum Circuit Simulation using Qiskit Aer

Quantum circuits can be simulated to analyse quantum protocols. Qiskit can be used to design quantum circuits and Qiskit Aer can be used as a simulation backend [6], [20]. This allows circuits to be run and analysed. This can be used to simulate preparing quantum states, changing bases, measurements and some noise models. The BB84 protocol can be simulated using quantum circuits. The classical part of the program will generate random bits, choose bases, perform basis reconciliation, key sifting and calculate the QBER. Changing the number of qubits sent or turning on an attack scenario will allow the experiment to be repeated [6], [20].

However the results will depend on how the quantum circuit is implemented, the random sampling, the simulation setup and any noise models turned on. An ideal simulation will not have all the features of a physical quantum channel. It cannot independently provide security for a working QKD system [4], [16]. QubiFide has been created using Python with Qiskit and Qiskit Aer. This provides a controlled environment to analyse BB84 and the intercept resend attack. The aim of QubiFide is to show how the main parts of the protocol work and how they relate to the application.

2.5 Classical Cryptography

The Quantum Key Distribution technique only provides a shared key however a traditional cryptographic scheme must be deployed to secure the actual messages being sent in [7], [8] a communication application.A well defined method can be found in RFC 5869 to derive cryptographic keys using the HKDF method.[7] This can be done using two different steps:

PRK = HKDF-Extract(salt, IKM)

OKM = HKDF-Expand(PRK, info, L)

Where (IKM) is the input keying material, PRK is the pseudo random key, (info) is additional context information and finally (L) is the desired length of the output. The HKDF method is simply key derivation and doesn't include error correction or privacy amplification which must be implemented in a full QKD protocol.

AES-GCM is an authenticated encryption mode provided by the National Institute of Standards and Technology [8]. When implemented properly this provides confidentiality and allows detection of any tampering with encrypted messages. Security relies on keys and nonces being handled correctly [8]..All these techniques are complimentary to each other within the application. BB84 provides a model for establishing keys, QBER calculates discrepancies between sampled values, HKDF provides key material for the application and AES-GCM provides protection for message contents. This shows how quantum cryptography can be investigated alongside traditional encryption within a software application [4], [18]..

Table 1: Comparative Summary of Existing Research

Author / Reference	Research Focus and Contribution	Relevance to QubiFide
Bennett and Brassard [1]	Introduced the BB84 quantum key distribution protocol using quantum states prepared in different bases.	Provides the foundation for the quantum key exchange simulation.
Lo and Chau [2]	Studied the unconditional security of quantum key distribution over long distances.	Provides theoretical background on QKD security.
Shor and Preskill [3]	Presented a simplified security proof for the BB84 protocol.	Supports the theoretical basis of the selected protocol.
Scarani et al. [4]	Reviewed security considerations for practical quantum key distribution systems.	Highlights the distinction between theoretical protocols and practical implementations.

Gisin et al. [9]	Reviewed the principles, applications, and challenges of quantum cryptography.	Provides background on quantum cryptography and its applications.
Fuchs et al. [12]	Investigated optimal eavesdropping strategies and information limits in quantum cryptography.	Helps explain how eavesdropping can affect quantum key exchange.
Krawczyk and Eronen [7]	Specified HKDF, a method for deriving cryptographic keys from input keying material.	Supports the application key derivation component.
Dworkin [8]	Documented Galois/Counter Mode (GCM) for authenticated encryption and authentication.	Provides the technical basis for AES-GCM message protection.
Qiskit Aer [6]	Provides tools for simulating quantum circuits and measurements.	Supports the software-based simulation of BB84 operations.
Lo, Curty, and Tamaki [17]	Reviewed developments and security considerations in quantum key distribution.	Provides broader context for evaluating the security limitations of the prototype.

2.7 Research Gap

There is existing research to provide a theory behind BB84 [1], [3]. There is a formal analysis of QKD security [2], [4], [14]. There are statistical methods to detect eavesdropping [5], [12] and software to simulate quantum circuits [6], [20]. There are also cryptographic standards to provide key derivation [7], [8]. All of these provide different parts to secure communications. They must be integrated with care. QubiFide aims to show this integration. This has been achieved by creating a web based prototype that includes simulated BB84 key distribution, QBER estimation, an optional intercept resend experiment and message encryption with authentication. This uses React and TypeScript for the frontend and Python FastAPI for the backend to manage the application. The project does not propose a new quantum protocol or encryption method. However it provides an interactive way to look at how basis selection, key sifting, measurement disturbance, QBER estimation and application encryption work together. Test results are needed to determine if the implementation works as intended.

The prototype is limited as it uses simulated quantum operations and the key exchange is for educational purposes. More development would be needed to include full QKD post processing, privacy amplification, authenticated protocol negotiation, improved identity management and a thorough security assessment [4], [17], [18]. The backend also has access to effective session keys so claims of end to end security cannot be made against a malicious server. Overall the literature reviewed shows that there is a mathematical and cryptographic background to QubiFide [3], [4], [7], [18].

3. PROBLEM DEFINITION

With the advances in digital communications there is an increased requirement for secure information. A major issue is securely distributing keys to participants wishing to communicate. Quantum Key Distribution (QKD) solves this problem by using quantum mechanics to share keys and detect interference [1] [3] [4].This project proposes a software solution that demonstrates how the BB84 protocol can be used for QKD and how it fits into traditional encryption. The problem being solved, goals and scope are below.

3.1 Problem Definition

Traditional cryptographic methods rely on securely maintained keys for securing communications. Creating and sharing keys via an insecure channel can cause problems. Tools are needed to help determine if interference occurs during key exchange.QubiFide solves this issue by providing a simulated demonstration of the BB84 protocol for quantum key distribution. Interception using an optional intercept resend attack is included along with QBER calculation and key validation to show any disturbances that may occur. Once a key has been validated the key material will be used to derive a key and messages can be encrypted using traditional cryptographic methods [1], [4], [5].

3.2 Project Scope

The aim of this project is to develop a communication system for two participants that includes temporary sessions, BB84 key exchange, eavesdropping and verification using QBER. This system makes use of quantum circuits implemented in a simulated format. The accepted keys are processed using HKDF, then can be used with AES-GCM encryption.This is currently a prototype for education and experimentation. It focuses mainly on demonstrating the intercept resend attack. Not all quantum attacks have been included within this project, nor has a formal proof of security. The post processing requirements are not fully included, so this framework should be seen as a demonstration of the principles of quantum assisted secure communication, rather than as a security solution [4] [18].

4. PROPOSED METHODOLOGY AND WORK DESCRIPTION

The methodology proposed will allow us to see how quantum key distribution can be combined with classical cryptography to create a secure communication between the two parties involved. The methodology contains creating a communication session between the two parties, creating quantum states, sending, measuring and creating a key. A main part of this methodology includes using the BB84 protocol to share keys. This protocol uses quantum states and different measurement bases to create the key. There is also a simulation for eavesdropping showing how eavesdropping would cause disturbances in the quantum states and therefore errors in the key [1] [3] [4].

The methodology contains several steps. Firstly there is establishing a communication session between the parties. The sender will create quantum states using random binary values and bases. The receiver will choose their own measurement bases. Once the quantum states have been sent the two parties will reconcile their bases to create a sifted key. This sifted key will then be tested for reliability using Quantum Bit Error Rate (QBER). If the requirements are met, the accepted key material will then be used to create a key for cryptography and this will be used to encrypt messages. This provides an understanding of how quantum key distribution can be combined with classical secure communications [4] [7] [8].

4.1 System Architecture

The system implements a modular client-server approach. This divides up tasks into multiple components making it easier to understand, maintain and scale. The main aspects that have been split up are user interaction, session management, quantum key distribution, verification and cryptography. Each component carries out its own task by communicating with other components.The overall system can be broken down into 5 main areas; User Interaction, Session Management, Quantum Key Distribution, Verification and Secure Communication.User Interaction: Enables users to interact with the communication system. Allows the user to set up a temporary communication session, giving them a session ID that they can share with the other party. The other party can then create a session using the session ID. When both parties are connected the user has the option to start the BB84 key exchange process. The interface also lets the user see how the key exchange is progressing, start the optional eavesdropper simulation, see the results of the QBER verification and send messages after a successful key exchange.

Session Management: Manages the communication between the two parties. This handles the information about sessions and what actions need to happen throughout a session. This ensures the key exchange is happening between the correct parties and the messages aren't sent until after verification has taken place. This also handles when different stages occur, such as creating a session, connecting to a session, key exchange, verification and secure messaging.
Quantum Key Distribution: This handles tasks involved with the BB84 protocol. This creates random binary values, bases for preparing the key, prepares the quantum states and simulates

sending them to the recipient. The recipient will choose bases for measuring the quantum states. The measurement results are then used to perform basis reconciliation, generating sifted keys. It also includes the optional intercept resend method, allowing the system to see how an eavesdropping attack would affect the quantum states being sent [1] [3].

Verification: This calculates the reliability of the sifted key by estimating the QBER. This compares a sample of the senders and receivers key bits to see how many are incorrect. This then calculates the error percentage between the two. The QBER value is then compared to a threshold value to see if it meets the acceptance criteria. If the verification is successful, the accepted key is then sent to the cryptographic stage. If not, the exchange will be rejected.

Secure Communication: This handles the accepted key to allow secure communication. This uses the HKDF method to derive a symmetric encryption key at the application level. The messages are then encrypted using AES-GCM. On the recipients end the encrypted data is handled using the relevant key. The authentication information is then checked, meaning the original message is displayed. This ensures messages are only sent after the key exchange has been verified [7] [8].

Overall, the components work together to form the QubiFide system. The user interface starts the processes, the session management handles the processes, the quantum component handles the key exchange, the verification determines if the generated key material is acceptable and the secure communication uses the accepted key material to securely send authenticated messages.

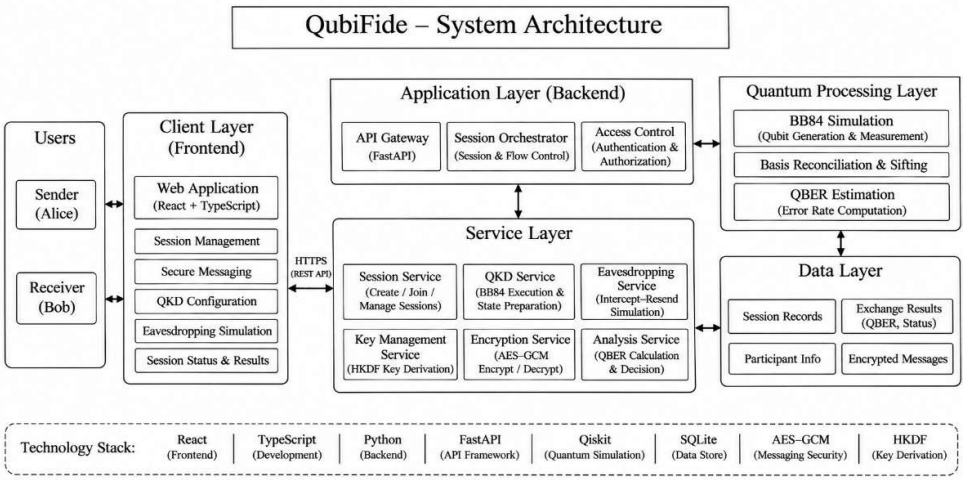


Fig 1. System architecture

4.2 BB84 based Key Generation

The first main aspect of this method is to generate a shared key using the BB84 Quantum Key Distribution protocol. This protocol was developed by Charles H. Bennett and Gilles Brassard in 1984 and takes advantage of the basic principles of quantum mechanics. With BB84, quantum states are prepared using different bases to generate a shared sequence of bits. Within the protocol it is based upon the principle that when a quantum state is measured using an incompatible basis uncertainty will occur and alter the original quantum state [1] [3].Within this scenario, it will be assumed that there are two parties, Alice and Bob. The first step is for Alice to generate a random series of binary values consisting of 0s and 1s. She will

also generate a random series of bases herself to prepare her quantum states. As BB84 uses two bases to prepare and measure the quantum states.

Computational Basis: These are the quantum states of $|0\rangle$ and $|1\rangle$, representing 0 and 1 respectively.

Hadamard Basis: These are the quantum states of $|+\rangle$ and $|-\rangle$, these are superposition states. Where:

$$|+\rangle = \frac{|0\rangle+|1\rangle}{\sqrt{2}} \tag{4.2.1}$$

$$|-\rangle = \frac{|0\rangle-|1\rangle}{\sqrt{2}} \tag{4.2.2}$$

Alice will prepare the appropriate quantum state depending on the binary value and the basis chosen. So if the computational basis is chosen, the value 0 will be represented by the quantum state $|0\rangle$ and the value 1 will be represented by $|1\rangle$. If the Hadamard basis is chosen, the values will be $|+\rangle$ and $|-\rangle$. This means that the random binary values are now quantum states.The quantum states are sent to Bob. Within this step, Bob will also generate a random series of bases himself, however he is unaware of the bases chosen by Alice. Bob will measure the quantum states using his chosen bases. If the basis chosen matches the one used by Alice, the result will be the expected binary value (ideal simulation). If the basis does not match then the result is probabilistic due to the incompatibility between the two bases. This is one of the fundamental principles behind BB84 being able to detect disturbances caused by an eavesdropper [1] [4].

Both parties will perform what is known as a Basis Reconciliation. They will compare the bases used for preparation and measurement via a classical communication channel. They do not share their actual binary values but only what bases were used. Any points where they used the same basis will be kept and any where the bases differ will be removed. This will leave them with a Sifted Key.This is the initial key that has been shared between the two parties using the BB84 protocol. However the key cannot be used to encrypt messages yet as it needs to be assessed for reliability. The generated sifted key will be passed onto the next part of the process, estimating and verifying the QBER. This shows how BB84 uses random bases and quantum measurements to create shared key material between two parties [1] [3] [4].

4.3 Detecting Eavesdropping and Estimating QBER

One of the key features of quantum key distribution is its ability to detect certain types of interference. Conventional communication systems could have the information copied or intercepted without changing the data. However, using the BB84 protocol measuring the quantum states using an incompatible basis can cause disturbances to the results at the receiver. QubiFide makes use of this feature to show how eavesdropping can be detected using an optional intercept resend attack [4] [5].Within this simulation, the eavesdropper will be represented by Eve. When enabling the eavesdropping feature, Eve will intercept the quantum states sent by Alice. She will prepare her own random bases and measure the quantum states.

Once she has measured the states she will prepare new quantum states based on the results and send these onto Bob. As Eve does not know what bases were used to prepare the quantum states by Alice, she might choose an incompatible basis.

When Eve measures the quantum states using the incorrect basis, there might be a difference between the outcome and what was originally encoded. This will cause Eve to prepare new quantum states to send, causing disturbances to Bob's results. This means that once the basis reconciliation has been complete, the sifted keys produced by Alice and Bob might have some differing bits. This indicates that there may have been disturbances during the quantum transmission.Ideally, if Eve was to perform an intercept resend attack on all the quantum states being sent and randomly chose between the two bases within BB84, the expected QBER would be around 25%. However, this may differ between simulations as there is a limited number of quantum states being sent and the results are probabilistic. [4] [5].To assess the key, Alice and Bob will choose a sample of bits from their sifted key. They will compare the values to estimate the error rate. Let the number of differing bits be represented by e and the total bits chosen be represented by k. The Quantum Bit Error Rate can be calculated using the equation below:

QBER = (e/k) x 100

This means that the QBER will show what percentage of the sample had differing bits. A low error rate means there were less differences seen. A high error rate means there were more differences between the bits. However, errors could also be caused by noise within the channel and imperfect devices. A high QBER indicates that there could have been some interference/disturbance during transmission [4] [16].The QBER will be compared against a set threshold. If the error rate is within an acceptable range then the key exchange will move onto the next stage. If the value is above the threshold then the key will be rejected. The sample of bits used will not be considered part of the secret key. This shows how the BB84 protocol uses statistical error estimation to assess the reliability of the generated key [3] [4] [5].

4.4 Key Derivation and Secure Communication

Following the verification of the sifted key, the approved portion is used to derive a symmetric key. This key will be used to encrypt messages sent between the users. QubiFide uses the HMAC based Key Derivation Function (HKDF) to produce the required key. The function is standardised and consists of an extraction and expansion process [7].The extraction process takes input keying material (IKM) and salt and outputs pseudorandom keying material (PRK). The expansion process then takes the PRK, additional information and output length to create the output keying material (OKM). These processes can be expressed using the equations:

PRK = HKDF-Extract(salt, IKM)

OKM = HKDF-Expand(PRK, info, L)

Where IKM is the approved input keying material, \(salt\) is the optional random value, \(PRK\) is the pseudorandom key produced, \(info\) is the additional information, \(L\) is the desired output length and OKM is the derived output keying material. The derived key is used within the traditional encryption function to encrypt messages.The messages then get encrypted using the Advanced Encryption Standard Galois Counter Mode (AES-GCM). This is an authenticated encryption scheme which provides confidentiality and integrity. The sender uses the derived symmetric key and nonce to encrypt their plaintext message. An encrypted message and authentication information is then produced [8].Bob can then use the corresponding derived key and information to decrypt and authenticate the message. The authentication process will confirm if the message has been tampered with or if the information is invalid. If the message is verified it will output the original plaintext message. Otherwise, if the message is not authenticated it will be rejected. The overall process shows the connection between quantum key exchange and traditional authenticated encryption. With the BB84 protocol generating the shared keying material, HKDF deriving the key to be used within the application and AES-GCM encrypting the messages. Therefore showing the implementation of quantum key distribution within a secure communication scenario [7] [8] [18].

4.5 Communication Session

As well as the encryption processes, the methodology also includes the management of communication sessions. This ensures that the key exchange and messages are sent between a specific set of users. This also provides a structure to the processes.The first user creates a temporary communication session. The system will generate a session ID which can be provided to the desired recipient. The second user can then use this ID to join the session. Once both users are connected the key exchange process can commence. This ensures that there is a specified context for the communication.

The users will then go through the quantum key generation and verification process. The system will store what stage of the key exchange they are on to ensure that messages cannot be sent until the key has been verified. The optional eavesdrop functionality can be engaged during the key exchange to see how the attempt to intercept affects the key and QBER. Whether the key is verified or not will determine what happens next.If the key exchange is approved the users can send encrypted messages. Each message will be encrypted using the derived symmetric key. They will also be authenticated before displaying at the receiving end. If the key exchange is not approved, the users will not be allowed to continue using the keying material. This provides a connection between the individual encryption processes.

4.6 Overview of the Process

Overall the entire QubiFide process involves a number of interconnected steps. After one person has created a communication session and provided the session ID to the intended recipient they join the session and begin the BB84 key exchange protocol. The sender begins by generating random binary values and bases to prepare quantum states that are sent to the recipient. The recipient will randomly choose bases to measure the quantum states.Once the quantum states have been measured both parties compare their chosen bases, keeping only the matching positions. This gives a sifted key which is used to calculate the QBER. A sample of

the bits are compared to calculate how many do not match. The intercept resend attack can also be turned on to show the effect of an eavesdropper.

Once the QBER has been calculated it can be compared to the threshold value. If the condition is met then the remaining key material will go through the key derivation stage using HKDF. This produces a symmetric key which will be used with AES-GCM to encrypt and authenticate messages between the two parties. The recipient will check the message using the relevant key and decrypt it.If the QBER is higher than the threshold value then the key material will not be accepted. If the message cannot be authenticated then the message will not be accepted. Overall the entire method can be broken down into creating a session, preparing, sending and measuring quantum states, comparing bases, generating a sifted key, calculating the QBER, accepting or rejecting the key, key derivation using cryptography and authenticating messages. This provides an interactive way to understand the basics of BB84 based quantum key distribution and how it can be integrated with traditional cryptographic methods. This project is designed to be a software prototype to simulate and educate on how quantum assisted communication works [1][4][7].

5. PROPOSED ALGORITHMS

This section details the algorithms involved within the quantum key distribution protocol. There are three algorithms which will be described throughout this section, these include BB84 key generation, eavesdropping detection using QBER calculation and secure messaging using traditional encryption. These algorithms allow us to understand the overall process performed by QubiFide to perform quantum key distribution and secure messaging [1] [4] [7].

5.1 BB84 Key Distribution

The BB84 key distribution algorithm generates a shared secret key between the sender (Alice) and receiver (Bob). Alice generates random bits of binary data and randomly picks between the computational basis and Hadamard basis to create a quantum state. This state is sent to Bob who randomly picks a measurement basis to measure the quantum state. Alice and Bob send their measurement basis to each other on a classical channel and discard any bits where there was no match in the measurement basis. This creates the sifted key. The eavesdropping process is optional and can be simulated by adding Eve into the system. She will measure the quantum state and send it to Bob. [1] [3] [4]

5.2 QBER Calculation

The calculated QBER allows us to check how reliable the sifted key is. Alice and Bob randomly choose some bits from their sifted key and send the values to each other on a classical channel. We record how many bits were different. This value is defined by:
e while the total number of compared bits is represented by k. We calculate the Quantum Bit Error Rate using the below equation:

$$QBER = \frac{e}{k} \times 100 \tag{5.2.1}$$

We then compare the QBER against a set threshold. If the error rate is within an acceptable range we can say the key exchange is fit for purpose. If it is out of range we reject the key. This shows how we can detect any disturbance that occurs due to an eavesdropper [4] [5].

5.3 Key Generation and Secure Messaging

The key generation and secure messaging algorithm uses the accepted key material to generate a symmetric key for secure messaging. Once we have confirmed the QBER is acceptable we take the shared key material and apply the HMAC based Key Derivation Function. We first run the HKDF-Extract function to generate a PRK. We then run the HKDF-Expand function to generate the encryption key (OKM). This key is then applied to the AES-GCM encryption algorithm and encrypts the senders message. The key is also used to generate authentication information. The receiver will use the correct key to decrypt the ciphertext and authenticate it. If the authentication passes we show the original message, otherwise the message is rejected.

This links the BB84 key generation with traditional authenticated encryption [7] [8]. The key generation can be seen below:

$$PRK = HKDF\text{-}Extract(salt, IKM)$$
$$OKM = HKDF\text{-}Expand(PRK, info, L)$$

These three algorithms demonstrate the overall process of QubiFide.

6. PROPOSED FLOWCHART / BLOCK DIAGRAM / DFD

In this section, an overview of the design of the proposed solution is presented. The key components of the system have been represented via a System Block Diagram, Workflow Diagram and Level-0 Data Flow diagrams (DFD). The diagrams give an idea of how data flows between the different modules and what are the major components. The system uses the BB84 protocol for Quantum Key Distribution and QBER analysis along with Key derivation and AES-GCM cipher to provide secure communication between the participants.

6.1 Proposed System Block Diagram

The block diagram depicts the overall setup of our QubiFide application. The frontend, written in React, allows users to participate in a session and send messages. We have a backend written in FastAPI for communication between modules. The BB84 module contains the simulation of the Quantum Key Exchange using Qiskit and Qiskit Aer. We analyse results by calculating how much QBER is present to identify if there was any eavesdropping. Once keys have been verified we use HKDF and AES-GCM to derive the keys and secure our messages. Finally, we have a local database to store session data/messages.

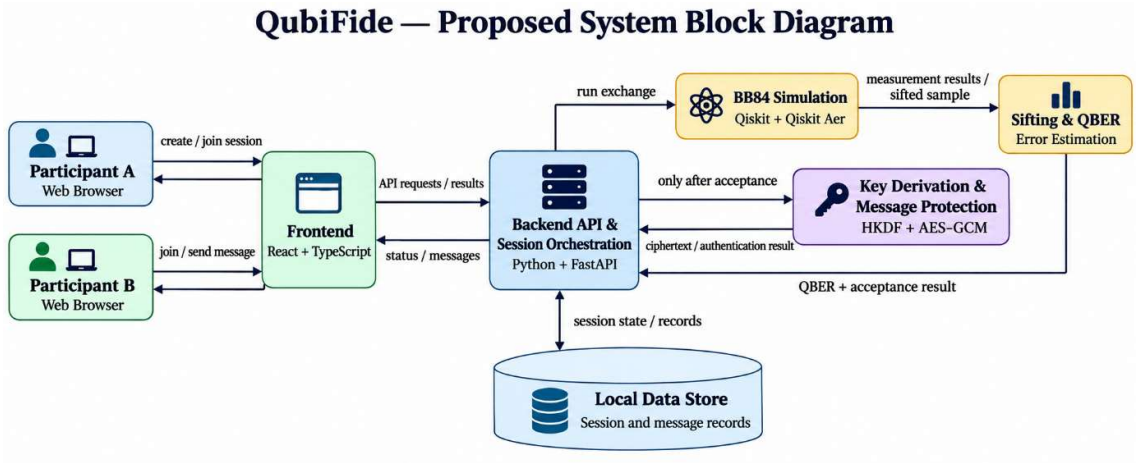


Fig 2. System block diagram

6.2 Workflow of the Proposed QubiFide System

The flowchart demonstrates the various processes within the QubiFide system. The process begins by creating a new session between two participants. Following this, random bits and bases are generated for the initialisation of the BB84 protocol. Quantum key exchange takes place followed by measurements. An optional eavesdropping attack can be simulated. Basis reconciliation takes place. The sifted key is calculated to find the QBER. If the acceptance conditions are satisfied the application key can be derived by using HKDF. The messages can now be securely encrypted using AES-GCM. However, if the acceptance conditions are not met then the key exchange will not be accepted and the messages can not be sent.

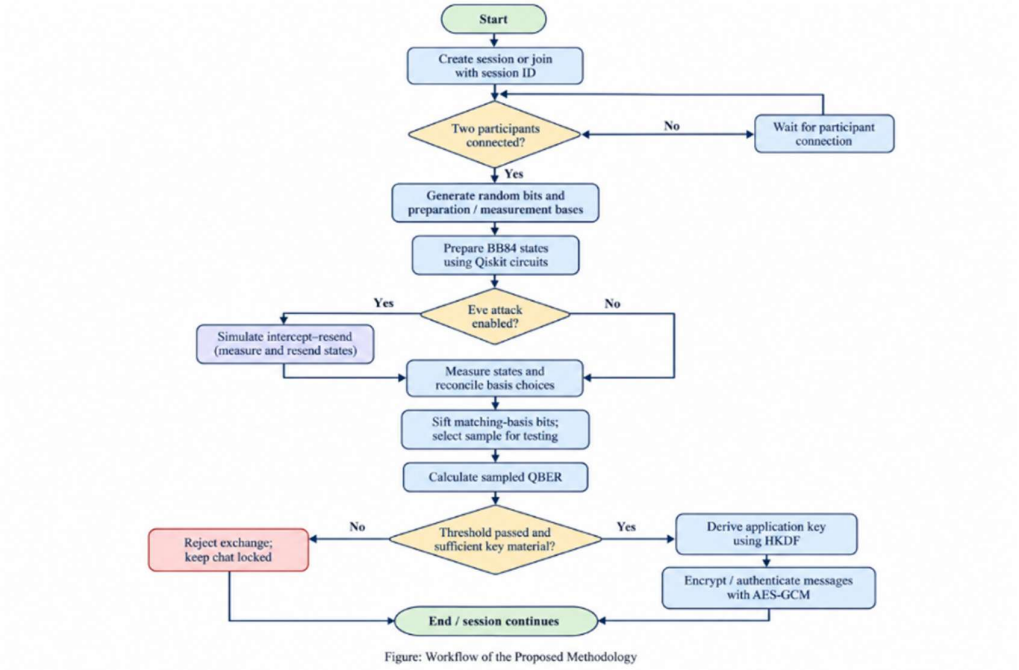


Fig 3. Workflow of system

6.3 Data Flow Diagram of the Proposed QubiFide System

This diagram demonstrates how information flows between the participants and main components of the system. There are a number of interactions between the Session Management Module and the Participants to allow them to create a session, participate in the communications and send messages. The BB84 Simulation and QBER Analysis Module manages the quantum key exchange, basis reconciliation and error estimation processes to ensure our key is secure. An optional eavesdropping module will simulate an intercept resend attack. Once the key exchange has been accepted by the user the Key Derivation and Message Protection Module will perform key derivation using the HKDF process and protect messages using AES-GCM. All encrypted messages, session information and related records are stored in the SQLite database. This diagram demonstrates how data flows between the modules.

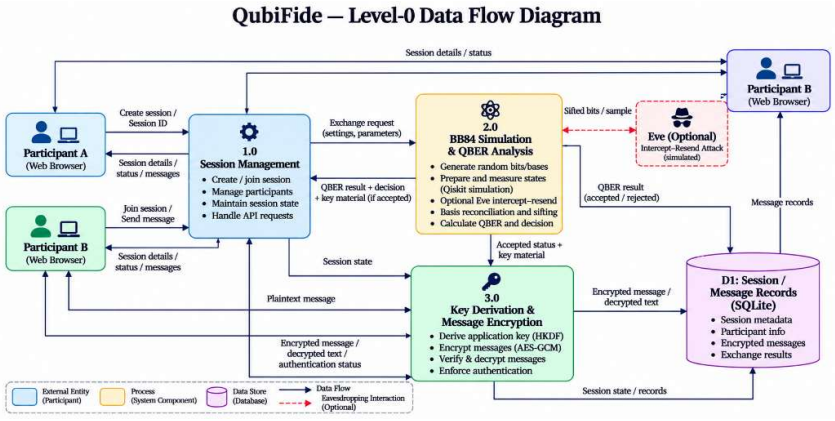


Fig 4. Data flow diagram

7. IMPLEMENTATION (TOOLS AND TECHNOLOGY USED)

The core of the QubiFide project was to develop a piece of software that provides an interactive communication platform for using quantum key distribution along with classical cryptography. The frontend of the application will provide the user interface, whilst the backend will handle the session functions, quantum simulation, key processing and message encryption. Throughout the project different technologies and libraries were utilised to complete the major parts of the application. This section outlines the development environment, technologies used, individual components and the testing processes.

7.1 Development Environment

The project uses a mixture of frontend and backend technologies. The frontend is built using React and Typescript with Vite used for development and building. The backend is built using Python and FastAPI to handle requests. Qiskit and Qiskit Aer are used for creating and simulating quantum circuits. The cryptography part of the application uses HKDF for key derivation and AES-GCM for message encryption with authentication. [6] [7] [8] [20]

Technology	Purpose
React	Development of the user interface
TypeScript	Frontend programming and type safety
Vite	Frontend development and build environment
Python	Backend development and quantum simulation
FastAPI	Backend API and session management
Qiskit	Creation and execution of quantum circuits
Qiskit Aer	Simulation of quantum circuits
HKDF	Cryptographic key derivation
AES-GCM	Message encryption and authentication

7.2 Frontend

The frontend is built using React and Typescript. React is used to create an interactive user interface. Typescript is used to make the code well structured by using static typing. Vite is used to provide a development and build environment for the frontend. The application provides an interactive session based communication platform. The user can create a session which can be shared with another participant. The second participant can then join the session using the unique identifier. Once both users have joined the session there are options to start and monitor the key exchange. The frontend shows the results of the quantum key distribution. This includes key exchange, QBER validation and eavesdropping (optional). Once the key exchange has been verified the user can send messages with encryption.

7.3 Backend

The backend was developed using Python and FastAPI. Python handles the application logic, quantum simulation and cryptography. FastAPI provides the Application Programming Interface. The backend handles key elements of the application including session creation, joining sessions, starting key exchange, key exchange results and message handling. The backend also handles the quantum simulation. The backend handles requests from the frontend, performs the relevant actions and returns the results.

7.4 Quantum Simulation

The quantum simulation component uses Qiskit, an open source quantum computing framework. Qiskit provides everything needed to create quantum circuits. Qiskit Aer provides a simulation environment so that the quantum circuits can be executed without a quantum machine. [6] [20] The quantum simulation component in QubiFide provides a representation of the quantum operations performed in the BB84 protocol. Quantum circuits are created to represent the state preparation and measurements using the computational and Hadamard bases. Measurement results are returned from the simulation. The component also provides an option to simulate the intercept resend attack. If this option is selected the quantum simulation will show the eavesdropper measuring and sending back the quantum states. Measurement results are returned.

7.5 Cryptography

The cryptography component handles the processing of the accepted key and provides a way to securely exchange messages. This component provides two main cryptography features, HKDF for key derivation and AES-GCM for authenticated encryption. HKDF is used to derive cryptographic key material for the application from the input keying material. This involves extracting and expanding the key to produce a suitable key for encryption. This follows a standardised key derivation process outlined in RFC 5869 [7]. AES-GCM is used to provide encryption and authentication. The sender will encrypt their plaintext message with the derived key. The receiver will verify and decrypt the received data using the corresponding key. If the data can be authenticated the original message is retrieved, otherwise it is rejected. This follows the authenticated encryption principles outlined in NIST SP 800-38D [8].

7.6 Frontend and Backend Integration

The frontend and backend are integrated using Application Programming Interface. The frontend will send requests to the backend to perform actions such as creating a session, joining a session, key exchange and sending messages. The backend will handle these requests. Whilst the session is running the backend will handle the quantum simulation and validation of the key. The frontend will show the results of the session. Once the key has been verified the message section will be enabled. Any messages sent will be handled by the cryptography component. The frontend, quantum simulation and encryption have been integrated into one application. Each component can run independently.

7.7 Testing

The application has been shown to perform its required functions by demonstrating that it can complete a secure key exchange and message transfer. Below are some screenshots that show the application has completed a successful key verification and message transfer.

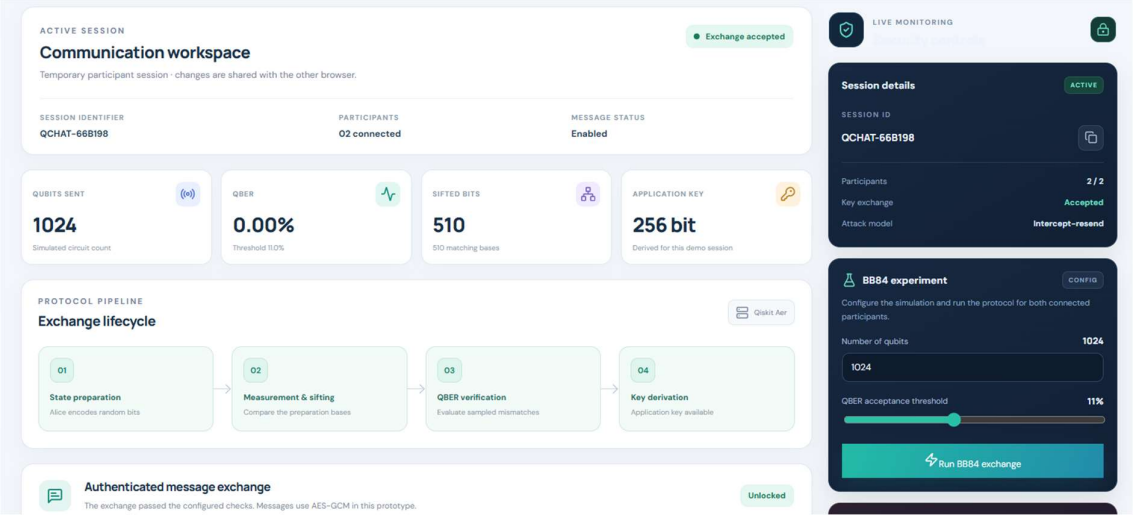


Figure 5: Completed Secure Key Exchange

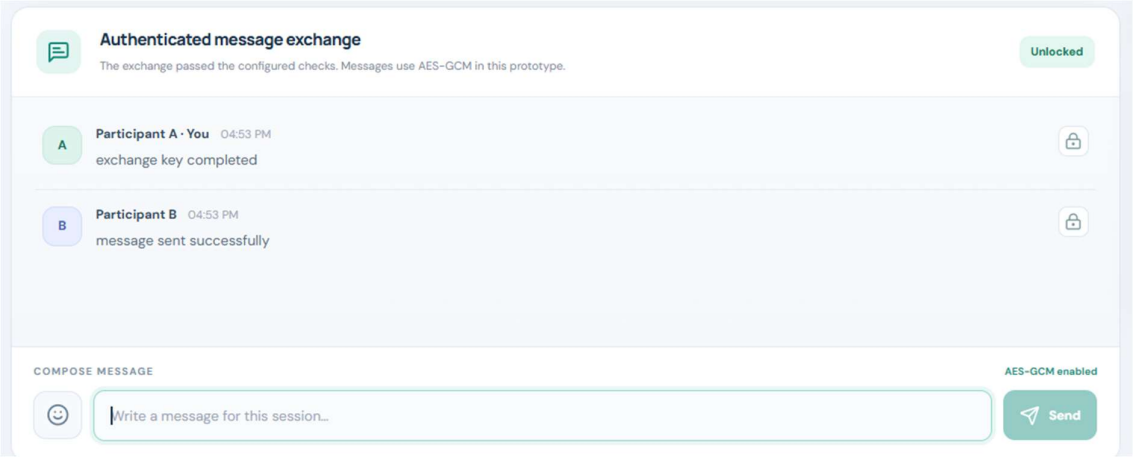


Figure 6: Completed Message Transfer

The screenshots show that the application was able to complete securely verify the key exchange and allow participants to exchange messages using the encryption method.

For eve interception the following figure shows the effect of intercepting and unable to exchange information after the attack.

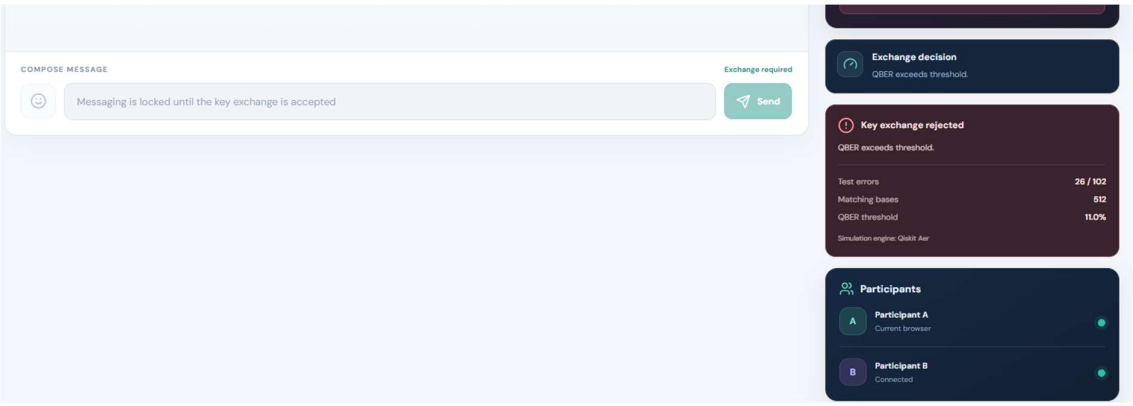


Fig 7: Eve interception

8. RESULT DISCUSSION AND ANALYSIS

Within QubiFide an implementation of BB84 Quantum Key Distribution, eavesdropping analysis and encryption of messages, using authenticated encryption have all been implemented within a web prototype. The simulation will generate quantum states, calculate the Quantum Bit Error Rate, perform analysis regarding the exchange based on the set acceptance criteria and allow messages to be sent. We will discuss the results in terms of the performance of the BB84 protocol, the accuracy of the QBER analysis and finally the correct encryption and authentication of the message. This will give us an insight into how well the different components have been implemented.

8.1 BB84 Quantum Key Distribution

A random set of bits and bases are generated. Each bit is then encoded into a quantum state using either the computational basis Z or Hadamard basis X. The receiver will then choose random bases to measure in. This will be done using Qiskit Aer.Once measured the bases are compared. Only those where the sender and receiver chose the same basis will remain. This gives us our sifted key[1][3].As the sender chose random bases to prepare with and the receiver randomly chose bases to measure with we would expect around 50% of the bits to have the same basis[1][4]. In an ideal scenario with no noise and no eavesdropping then the bits should match up. We can record the qubit amount, number of matching bases and number of remaining bits.

If eavesdropping is switched on then an intercept resend attack will occur. Eve will measure the intercepted quantum states using a random basis. She will then prepare new states to send to the receiver. Due to measuring in an incorrect basis then this can cause disturbance to the quantum states. This means there will be mismatched bits in the sifted key. This gives us the basis for the QBER analysis [4][5][12].

8.2 Quantum Bit Error Rate Analysis

The Quantum Bit Error Rate tells us the percentage of mismatched bits in our sample. The formula for the QBER is:

$$\hat{Q} = \frac{e}{k} \times 100\% \tag{8.2.1}$$

Where,

e = Number of mismatched bits
k = Total number of bits compared
$\hat{Q}$ = Estimated QBER (%)

A low QBER means there were less mismatches in the sample whilst a high QBER means there were more mismatches. We then compare this to our threshold to see if the exchange has been accepted.Under normal ideal conditions we would expect around 25% QBER in the sifted key due to an intercept resend attack with random bases. However due to the random nature of this there may be variation in the results. We should consider this an expected value [4], [5].

.

Table 2: QBER-Based Eavesdropping Analysis and Exchange Verification

Test condition	Expected behaviour	Verification outcome
Eavesdropping disabled	Zero or low QBER in an ideal noiseless simulation	Secure key exchange
Eavesdropping enabled	Increased probability of mismatches	Eavesdropping detected
QBER within the configured threshold	Exchange may be accepted if other conditions are satisfied	Exchange accepted
QBER above the configured threshold	Exchange is rejected according to the implemented rule	Exchange rejected

Looking at the first two rows we can look to see if the results match what we expect to happen with an intercept resend attack. Looking at the last two rows we can see if the application is correctly applying the acceptance criteria. Where possible we should try to run this multiple times to get an accurate idea of what is happening with the random bases and measurements [4], [5].

8.3 Exchange Acceptance

The QBER analysis will help us decide whether to accept or reject the key exchange. If the estimated QBER is above our threshold then we should reject the exchange. The application should also ensure that messages cannot be sent if the exchange has been rejected or there is not enough candidate key material.This is important to ensure that we are not just getting a numerical value for our QBER but that the implementation is also ensuring that there is enough key material to continue with the exchange [4][17].

Table 3: Key Exchange Verification and Acceptance

Scenario	Expected system response	Verification outcome
Exchange satisfies all configured conditions	Accept exchange and permit the next stage	Key exchange accepted
QBER exceeds the threshold	Reject exchange	Exchange rejected due to high QBER
Insufficient candidate key material	Prevent key derivation and messaging	Key derivation blocked
Exchange has been rejected	Keep messaging disabled	Messaging remains disabled

This helps us understand if the verification is working correctly.

8.4 Secure Message Encryption & Authentication

Once the exchange has been accepted the application will derive an encryption key using HKDF. The message will then be encrypted using AES-GCM giving a tag. The receiver will then perform verification using the correct key, nonce, ciphertext and tag. We can look to see if we can retrieve the original message and if any changes can be detected in the protected message data [8].

Table 4: Secure Message Encryption and Authentication Testing

Test case	Expected behaviour	System test results
Valid message encryption and decryption	Original plaintext is recovered	Message successfully encrypted and decrypted.
Modified ciphertext	Authentication fails and the message is rejected	Modified ciphertext rejected.
Modified authentication tag	Authentication fails and the message is rejected	Authentication failure detected.
Messaging after a rejected exchange	Messaging remains disabled	Messaging access remains disabled.
Attempt to proceed without sufficient key material	Key-dependent messaging is blocked	Key derivation blocked due to insufficient key material.

If the original message is retrieved this shows the encryption and decryption are compatible with each other. If the ciphertext or tag is incorrect then we know the authentication is working [8].This looks at a different aspect to the QBER analysis. The QBER analysis looked at estimating how many mismatches occurred during the key exchange. Whilst the AES-GCM authentication looks at verifying that the encrypted message data is intact and authentic. Just because the message was authenticated doesn't mean the previous key exchange is secure [4].

8.5 Discussion & Comparison

The expected outcome of QubiFide will depend on how the main components interact. The BB84 simulation will create the candidate key, we will then sift the key to keep only those with matching bases. The QBER analysis will look at a sample of the data to see how many mismatches there are. Finally the exchange verification will determine if we can continue with deriving the key and encrypting messages.With no eavesdropping we would generally expect the QBER to be 0% as the states have been prepared, measured and compared correctly. With eavesdropping switched on we can expect more mismatches due to the interference caused by measuring [4][5].Comparing the two will give us a good idea of how the simulation is performing.

However we cannot guarantee that there was no eavesdropping just because we have a low QBER. The attack may not have been detected within the sample. Especially if the sample size is low. We should take this into consideration along with the sampling, attack, assumptions and threshold.The cryptography experiments give us further evidence of how the application is operating. The application can derive keys using HKDF [7]. The messages can also be

encrypted using authenticated encryption (AES-GCM). The whole process shows how a simulated key exchange could be connected to a communications interface. However it is just evidence of how the implementation is performing.

9. CONCLUSION AND FUTURE SCOPE

This project demonstrates the concept of quantum key distribution along with classical cryptography. The system uses various functions including the BB84 protocol, eavesdropping example, QBER calculations, key derivation and authentication. This section discusses the results of the project, any limitations and potential areas of improvement.

9.1 Conclusion

The project has successfully demonstrated the basic process of the BB84 protocol. Using the Qiskit Aer simulator the system allows two parties to generate a key using quantum states and different bases. An optional eavesdropping attack can be utilised to show how errors can be introduced.The system uses the Quantum Bit Error Rate (QBER) to calculate if there was any interference and whether the generated key should be accepted. If the key has been verified then it will use HKDF to derive the key. The information can then be encrypted and authenticated using AES-GCM [5][7][8].Overall this system provides an interactive way to understand how QKD works and how it can be combined with classical encryption. However this is just a simulated prototype [4][6].

9.2 Limitations

This system does have some limitations. Whilst the BB84 protocol can be simulated using the Qiskit Aer simulator, there are no physical quantum components being used. Therefore any real-world issues such as losses and imperfections in the hardware cannot be fully represented [4][6].Whilst the intercept resend attack can be demonstrated, the system does not demonstrate every possible quantum attack that can occur. The QBER cannot be fully calculated as it is only using a limited sample size so can vary between simulations. There is also not a full proof of security or all post processing steps [4][12][18].Some parts of the session and key generation have been implemented in the backend so the prototype is not completely secure end to end.

9.3 Future Scope

This system could be improved by adding channel imperfections such as noise, losses and imperfections in the detectors. More eavesdropping strategies could also be added to see what effect they have on the QBER and key [4][16].Further developments could include information reconciliation, privacy amplification, finite key analysis and more secure authentication methods. The system could also be adapted to allow for multi-party [3][17][18].If this system could be integrated into actual quantum hardware then the results could be compared. These improvements could make this a more complete platform to study QKD.

10. REFERENCES

1. C. H. Bennett and G. Brassard, “Quantum cryptography: Public key distribution and coin tossing,” in *Proc. IEEE International Conference on Computers, Systems and Signal Processing*, Bangalore, India, 1984, pp. 175–179.

2. H.-K. Lo and H. F. Chau, “Unconditional security of quantum key distribution over arbitrarily long distances,” *Science*, vol. 283, no. 5410, pp. 2050–2056, 1999, doi: 10.1126/science.283.5410.2050.

3. P. W. Shor and J. Preskill, “Simple proof of security of the BB84 quantum key distribution protocol,” *Physical Review Letters*, vol. 85, no. 2, pp. 441–444, 2000, doi: 10.1103/PhysRevLett.85.441.

4. V. Scarani, H. Bechmann-Pasquinucci, N. J. Cerf, M. Dušek, N. Lütkenhaus, and M. Peev, “The security of practical quantum key distribution,” *Reviews of Modern Physics*, vol. 81, no. 3, pp. 1301–1350, 2009, doi: 10.1103/RevModPhys.81.1301.

5. C. Lee, I. Sohn, and W. Lee, “Eavesdropping detection in BB84 quantum key distribution protocols,” *IEEE Transactions on Network and Service Management*, vol. 19, no. 3, pp. 2689–2701, 2022, doi: 10.1109/TNSM.2022.3165202.

6. IBM Quantum, “Qiskit Aer documentation.” [Online]. Available: https://qiskit.github.io/qiskit-aer/. [Accessed: Oct. 25, 2026].

7. H. Krawczyk and P. Eronen, “HMAC-based extract-and-expand key derivation function (HKDF),” RFC 5869, Internet Engineering Task Force, May 2010. [Online]. Available: https://www.rfc-editor.org/rfc/rfc5869.

8. M. Dworkin, *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*, NIST Special Publication 800-38D, Nov. 2007, doi: 10.6028/NIST.SP.800-38D.

9. N. Gisin, G. Ribordy, W. Tittel, and H. Zbinden, “Quantum cryptography,” *Reviews of Modern Physics*, vol. 74, no. 1, pp. 145–195, 2002, doi: 10.1103/RevModPhys.74.145.

10. A. K. Ekert, “Quantum cryptography based on Bell’s theorem,” *Physical Review Letters*, vol. 67, no. 6, pp. 661–663, 1991, doi: 10.1103/PhysRevLett.67.661.

11. C. H. Bennett, G. Brassard, and N. D. Mermin, “Quantum cryptography without Bell’s theorem,” *Physical Review Letters*, vol. 68, no. 5, pp. 557–559, 1992, doi: 10.1103/PhysRevLett.68.557.

12. C. A. Fuchs, N. Gisin, R. B. Griffiths, C.-S. Niu, and A. Peres, “Optimal eavesdropping in quantum cryptography. I. Information bound and optimal strategy,” *Physical Review A*, vol. 56, no. 2, pp. 1163–1172, 1997, doi: 10.1103/PhysRevA.56.1163.
13. N. Lütkenhaus, “Security against individual attacks for realistic quantum key distribution,” *Physical Review A*, vol. 61, no. 5, Art. no. 052304, 2000, doi: 10.1103/PhysRevA.61.052304.

14. D. Mayers, “Unconditional security in quantum cryptography,” *Journal of the ACM*, vol. 48, no. 3, pp. 351–406, 2001, doi: 10.1145/382780.382781.

15. W.-Y. Hwang, “Quantum key distribution with high loss: Toward global secure communication,” *Physical Review Letters*, vol. 91, no. 5, Art. no. 057901, 2003, doi: 10.1103/PhysRevLett.91.057901.

16. D. Gottesman, H.-K. Lo, N. Lütkenhaus, and J. Preskill, “Security of quantum key distribution with imperfect devices,” *Quantum Information and Computation*, vol. 4, no. 5, pp. 325–360, 2004. [Online]. Available: https://preskill.caltech.edu/pubs/preskill-2004-imperfect.pdf.

17. H.-K. Lo, M. Curty, and K. Tamaki, “Secure quantum key distribution,” *Nature Photonics*, vol. 8, pp. 595–604, 2014, doi: 10.1038/nphoton.2014.149.

18. S. Pirandola *et al*., “Advances in quantum cryptography,” *Advances in Optics and Photonics*, vol. 12, no. 4, pp. 1012–1236, 2020, doi: 10.1364/AOP.361502.

19. N. Lütkenhaus, “Estimates for practical quantum cryptography,” *Physical Review A*, vol. 59, no. 5, pp. 3301–3319, 1999, doi: 10.1103/PhysRevA.59.3301.

20. G. Aleksandrowicz *et al*., “Qiskit: An open-source framework for quantum computing,” Zenodo, version 0.7.2, Jan. 23, 2019, doi: 10.5281/zenodo.2562111.

21. C. H. Bennett, “Quantum cryptography using any two nonorthogonal states,” *Physical Review Letters*, vol. 68, no. 21, pp. 3121–3124, 1992, doi: 10.1103/PhysRevLett.68.3121.